

Lab Assignment

C17 — STL `set<T>` — Core Operations

insert • emplace • erase • find • count • contains • clear • size • empty • iterate • set operations

1. Objective

This lab explores the core operations of the STL `set<T>` container. A set stores a collection of **unique** elements in **sorted order**. It automatically rejects duplicate values and keeps all elements sorted at all times, making it ideal for membership testing, deduplication, and maintaining ordered collections.

All experiments use Springfield Power Plant characters as data.

By the end of this lab you will be able to:

- ▶ Define and populate a `set<string>` and `set<int>` using initializer lists and `insert()`.
- ▶ Understand why duplicate values are silently ignored.
- ▶ Traverse a set using range-for and forward/reverse iterators.
- ▶ Search for elements using `find()`, `count()`, and `contains()`.
- ▶ Remove elements using `erase()` by value and by iterator.
- ▶ Clear all elements with `clear()` and check emptiness with `empty()`.
- ▶ Perform set operations: union, intersection, and difference.

2. Background

A set has three defining properties that distinguish it from a vector or map:

- ▶ **Uniqueness** — every element must be distinct. Inserting a value that already exists is silently ignored. No error is thrown.
- ▶ **Sorted order** — elements are always stored in ascending order, regardless of insertion order. Iterating a set always yields elements from smallest to largest.
- ▶ **No direct access** — sets do not support `operator[]` or `at()`. You cannot access the *n*th element directly. Use `find()` or `iterate`.

Internally a set uses a balanced binary tree, giving $O(\log n)$ time for insert, erase, and find. For faster average-case lookups when order does not matter, use `unordered_set` instead.

3. Quick Reference – Set Methods

Method	Description
s.insert(val)	Inserts val. Ignored if val already exists. Returns pair<iterator, bool>.
s.emplace(val)	Constructs val in place. Same behavior as insert on duplicate.
s.erase(val)	Removes the element with that value. Returns number erased (0 or 1).
s.erase(iterator)	Removes the element pointed to by the iterator.
s.find(val)	Returns iterator to val, or end() if not found.
s.count(val)	Returns 1 if val exists, 0 if not.
s.contains(val)	Returns true if val exists. C++20 — cleaner than count().
s.clear()	Removes all elements. size() becomes 0.
s.size()	Returns the number of elements.
s.empty()	Returns true if size() == 0.
s.begin()	Iterator to the smallest element.
s.end()	Iterator to one past the largest element.
s.rbegin()	Reverse iterator to the largest element.
s.rend()	Reverse iterator to one before the smallest element.
s.lower_bound(val)	Iterator to first element >= val.
s.upper_bound(val)	Iterator to first element > val.

set<T> vs vector<T> vs map<K,V> at a glance:

Feature	vector<T>	set<T>	map<K,V>
Duplicates allowed	Yes	No	No (keys unique)
Sorted automatically	No	Yes	Yes (by key)
Direct index access	Yes (v[i])	No	Yes (m[key])
Search time	O(n)	O(log n)	O(log n)
Best for	Sequences	Unique items	Key-value lookup

4. Required Experiments

Experiment 01 — Define, Populate, and Observe Uniqueness

Demonstrate that a set automatically rejects duplicates and stores elements in sorted order.

Tasks:

- ▶ Create `nameSet` using an initializer list with five Springfield names.
- ▶ Add three more names using `insert()` and `emplace()`.
- ▶ Attempt to insert a name already in the set — confirm it is silently ignored.
- ▶ Check the return value of `insert()` to see whether insertion succeeded.
- ▶ Print the set and confirm entries appear in alphabetical order.
- ▶ Create `ageSet = { 45, 10, 42, 8, 104, 10, 45 }` and confirm duplicates are removed.

```
void experiment01() {
    cout << "=== Experiment 01: Define, Populate, Uniqueness ===\n";

    // Initializer list -- sorted automatically
    set<string> nameSet = {
        "Homer Simpson",
        "Marge Simpson",
        "Bart Simpson",
        "Lisa Simpson",
        "Maggie Simpson",
    };

    // insert()
    nameSet.insert("Ned Flanders");

    // emplace()
    nameSet.emplace("Montgomery Burns");

    // Duplicate -- silently ignored
    auto result = nameSet.insert("Homer Simpson");
    cout << "insert(Homer again) succeeded: "
         << (result.second ? "YES" : "NO -- already exists") << "\n";

    cout << "nameSet size=" << nameSet.size() << " (sorted):\n";
    for (const auto& name : nameSet)
        cout << " " << name << "\n";

    // set<int> -- duplicates removed, sorted automatically
    set<int> ageSet = { 45, 10, 42, 8, 104, 10, 45 };
    cout << "\nageSet (dups removed, sorted): [ ";
    for (int a : ageSet) cout << a << " ";
    cout << "]" << " size=" << ageSet.size() << "\n";
}
```

Experiment 02 — Traverse: Range-for and Iterators

A set always delivers elements in sorted order. Both forward and reverse traversal demonstrate this.

Tasks:

- Traverse using a range-for loop — elements arrive in ascending order.
- Traverse using a forward iterator with `begin()` and `end()`.
- Traverse in reverse using `rbegin()` and `rend()` — elements arrive in descending order.
- Note: you CANNOT modify an element through a set iterator — all iterators are effectively `const`.

```
void experiment02() {
    cout << "=== Experiment 02: Traversal ===\n";

    set<string> nameSet = {
        "Homer Simpson", "Marge Simpson", "Bart Simpson",
        "Lisa Simpson", "Montgomery Burns",
    };

    // Style 1 -- range-for (ascending order)
    cout << "Range-for (A to Z):\n";
    for (const auto& name : nameSet)
        cout << " " << name << "\n";

    // Style 2 -- forward iterator
    cout << "\nForward iterator:\n";
    for (auto it = nameSet.begin(); it != nameSet.end(); ++it)
        cout << " " << *it << "\n";

    // Style 3 -- reverse iterator (descending order)
    cout << "\nReverse iterator (Z to A):\n";
    for (auto it = nameSet.rbegin(); it != nameSet.rend(); ++it)
        cout << " " << *it << "\n";

    // Note: *it = "new value"; would be a COMPILE ERROR
    // Set elements are immutable through iterators
}
```

Experiment 03 — Search: find(), count(), contains()

Three ways to test membership in a set — each with slightly different syntax and return type.

Tasks:

- ▶ Use `find()` to search for Homer — confirm the iterator is not `end()`.
- ▶ Use `find()` to search for Krusty — confirm it returns `end()`.
- ▶ Use `count()` to check membership — returns 1 (found) or 0 (not found).
- ▶ Use `contains()` (C++20) for the most readable membership test.
- ▶ Use `lower_bound()` and `upper_bound()` to find the insertion range for a value.

```
void experiment03() {
    cout << "=== Experiment 03: Search ===\n";

    set<string> nameSet = {
        "Homer Simpson", "Marge Simpson", "Bart Simpson",
        "Lisa Simpson", "Montgomery Burns",
    };

    // find() -- returns iterator or end()
    auto it = nameSet.find("Homer Simpson");
    if (it != nameSet.end())
        cout << "find(Homer): FOUND -> " << *it << "\n";
    else
        cout << "find(Homer): NOT FOUND\n";

    it = nameSet.find("Krusty the Clown");
    cout << "find(Krusty): "
        << (it != nameSet.end() ? "FOUND" : "NOT FOUND (expected)") << "\n";

    // count() -- returns 0 or 1
    cout << "count(Homer): " << nameSet.count("Homer Simpson") << "\n";
    cout << "count(Krusty): " << nameSet.count("Krusty the Clown") << "\n";

    // contains() -- C++20, most readable
    cout << "contains(Bart): "
        << (nameSet.contains("Bart Simpson") ? "YES" : "NO") << "\n";
    cout << "contains(Ned): "
        << (nameSet.contains("Ned Flanders") ? "YES" : "NO") << "\n";

    // lower_bound and upper_bound on a set<int>
    set<int> ages = { 8, 10, 42, 45, 60, 104 };
    auto lb = ages.lower_bound(40); // first element >= 40
    auto ub = ages.upper_bound(60); // first element > 60
    cout << "\nAges >= 40: ";
    for (auto i = lb; i != ages.end(); ++i) cout << *i << " ";
    cout << "\nAges in (40, 60]: ";
    for (auto i = lb; i != ub; ++i) cout << *i << " ";
    cout << "\n";
}
```

Experiment 04 — erase() and clear()

`erase()` removes an element by value or by iterator. Unlike vector, erasing from a set does not invalidate other iterators. `clear()` removes all elements at once.

Tasks:

- ▶ Erase an element by value using `erase("Bart Simpson")`. Confirm with `count()`.
- ▶ Erase an element by iterator: find Homer then pass the iterator to `erase()`.
- ▶ Erase all ages below 18 from a `set<int>` using an iterator loop.
- ▶ Call `clear()` and verify `size()` is 0 and `empty()` returns true.

```
void experiment04() {
    cout << "=== Experiment 04: erase() and clear() ===\n";

    set<string> nameSet = {
        "Homer Simpson", "Marge Simpson", "Bart Simpson",
        "Lisa Simpson", "Maggie Simpson", "Montgomery Burns",
    };
    cout << "Initial size: " << nameSet.size() << "\n";

    // Erase by value
    nameSet.erase("Bart Simpson");
    cout << "After erase(Bart): size=" << nameSet.size()
        << " Bart present: " << (nameSet.count("Bart Simpson") ? "YES" : "NO")
        << "\n";

    // Erase by iterator
    auto it = nameSet.find("Homer Simpson");
    if (it != nameSet.end())
        nameSet.erase(it);
    cout << "After erase(Homer by iterator): size=" << nameSet.size() << "\n";

    // Remaining names
    cout << "Remaining: ";
    for (const auto& n : nameSet) cout << n << " ";
    cout << "\n";

    // Erase all ages below 18 from a set<int>
    set<int> ages = { 1, 8, 10, 42, 45, 60, 104 };
    cout << "\nAges before: ";
    for (int a : ages) cout << a << " ";
    cout << "\n";

    for (auto it2 = ages.begin(); it2 != ages.end(); ) {
        if (*it2 < 18)
            it2 = ages.erase(it2); // returns iterator to next element
        else
            ++it2;
    }
    cout << "Ages after erase(<18): ";
    for (int a : ages) cout << a << " ";
    cout << "\n";

    // clear()
    nameSet.clear();
    cout << "After clear(): size=" << nameSet.size()
        << " empty=" << (nameSet.empty() ? "true" : "false") << "\n";
}
```

Experiment 05 — Set Operations: Union, Intersection, Difference

The STL provides three algorithms in `<algorithm>` that implement classic set theory operations. Each takes two sorted ranges and writes results to an output iterator.

Three set operations:

- ▶ **set_union(A, B, out)** — all elements that are in A OR B (or both). Duplicates appear once.
- ▶ **set_intersection(A, B, out)** — only elements that are in BOTH A AND B.
- ▶ **set_difference(A, B, out)** — elements in A but NOT in B.

Tasks:

- ▶ Create `setA` = day-shift employees and `setB` = union members.
- ▶ Compute and print the union of A and B.
- ▶ Compute and print the intersection of A and B.
- ▶ Compute and print the difference A – B (day-shift workers who are NOT union members).

```
#include <algorithm>    // set_union, set_intersection, set_difference
#include <iterator>     // inserter

void experiment05() {
    cout << "=== Experiment 05: Set Operations ===\n";

    // setA -- day-shift employees
    set<string> setA = {
        "Homer Simpson",
        "Lenny Leonard",
        "Carl Carlson",
        "Frank Grimes",
        "Barney Gumble",
    };

    // setB -- union members
    set<string> setB = {
        "Homer Simpson",
        "Lenny Leonard",
        "Moe Szyslak",
        "Barney Gumble",
        "Otto Mann",
    };

    // Union -- A OR B
    set<string> unionSet;
    set_union(setA.begin(), setA.end(),
              setB.begin(), setB.end(),
              inserter(unionSet, unionSet.begin()));
    cout << "Union (A OR B):\n";
    for (const auto& n : unionSet) cout << "  " << n << "\n";

    // Intersection -- A AND B
    set<string> interSet;
    set_intersection(setA.begin(), setA.end(),
                    setB.begin(), setB.end(),
                    inserter(interSet, interSet.begin()));
    cout << "\nIntersection (A AND B):\n";
```

```

    for (const auto& n : interSet) cout << "  " << n << "\n";

    // Difference -- A minus B
    set<string> diffSet;
    set_difference(setA.begin(), setA.end(),
                  setB.begin(), setB.end(),
                  inserter(diffSet, diffSet.begin()));
    cout << "\nDifference (A minus B -- day-shift, non-union):\n";
    for (const auto& n : diffSet) cout << "  " << n << "\n";
}

```

Experiment 06 — Practical Application: Deduplication

One of the most common real-world uses of a set is deduplication — removing repeated values from a sequence. This experiment reads a list of department names that contains duplicates and uses a set to produce a unique sorted list.

Tasks:

- ▶ Start with a vector of department names that contains many duplicates.
- ▶ Insert all values into a `set<string>` — duplicates are automatically discarded.
- ▶ Print the deduplicated, sorted result.
- ▶ Count how many unique departments exist vs the original list length.
- ▶ Build a second set of "critical" departments and use `set_intersection` to find which critical departments are actually staffed.

```

void experiment06() {
    cout << "=== Experiment 06: Deduplication ===\n";

    // Raw department list with duplicates
    vector<string> raw = {
        "Sector 7-G", "Maintenance", "Sector 7-G", "Engineering",
        "Safety",      "Sector 7-G", "Maintenance", "Research",
        "Safety",      "Engineering", "Executive",   "Maintenance",
        "Sector 7-G", "IT",           "Research",   "Safety",
        "Executive",  "IT",           "Maintenance", "Sector 7-G",
    };
    cout << "Raw list size (with duplicates): " << raw.size() << "\n";

    // Insert all into a set -- duplicates silently discarded
    set<string> deptSet(raw.begin(), raw.end());
    cout << "Unique departments: " << deptSet.size() << "\n";
    for (const auto& d : deptSet)
        cout << "  " << d << "\n";

    // Critical departments that MUST be staffed
    set<string> critical = {
        "Engineering", "Safety", "Sector 7-G", "Security",
    };

    // Which critical departments are actually staffed?
    set<string> staffedCritical;
    set_intersection(deptSet.begin(), deptSet.end(),
                    critical.begin(), critical.end(),
                    inserter(staffedCritical, staffedCritical.begin()));
}

```



```

    cout << "\nCritical departments that are staffed:\n";
    for (const auto& d : staffedCritical)
        cout << "    " << d << "\n";

    // Which critical departments are NOT staffed?
    set<string> unstaffed;
    set_difference(critical.begin(), critical.end(),
                  deptSet.begin(), deptSet.end(),
                  inserter(unstaffed, unstaffed.begin()));
    cout << "\nCritical departments NOT staffed:\n";
    if (unstaffed.empty())
        cout << "    All critical departments are staffed.\n";
    else
        for (const auto& d : unstaffed) cout << "    " << d << "\n";
}

```

main() — Call All Experiments

```

#include <iostream>
#include <set>
#include <vector>
#include <string>
#include <algorithm>
#include <iterator>
using namespace std;

int main() {
    experiment01();
    experiment02();
    experiment03();
    experiment04();
    experiment05();
    experiment06();
    cout << "\nAll done!\n";
    return 0;
}

```

5. Expected Output (partial)

```
=== Experiment 01: Define, Populate, Uniqueness ===
insert(Homer again) succeeded: NO -- already exists
nameSet size=7 (sorted):
  Bart Simpson
  Homer Simpson
  Lisa Simpson
  Maggie Simpson
  Marge Simpson
  Montgomery Burns
  Ned Flanders
ageSet (dups removed, sorted): [ 8 10 42 45 104 ] size=5

=== Experiment 02: Traversal ===
Range-for (A to Z):
  Bart Simpson
  Homer Simpson ...

=== Experiment 03: Search ===
find(Homer): FOUND -> Homer Simpson
find(Krusty): NOT FOUND (expected)
count(Homer): 1
count(Krusty): 0
contains(Bart): YES
contains(Ned): NO
Ages >= 40: 42 45 60 104
Ages in (40, 60]: 42 45 60

=== Experiment 04: erase() and clear() ===
Initial size: 6
After erase(Bart): size=5 Bart present: NO
After erase(Homer by iterator): size=4
Remaining: Lisa Simpson Maggie Simpson Marge Simpson Montgomery Burns
Ages before: 1 8 10 42 45 60 104
Ages after erase(<18): 42 45 60 104
After clear(): size=0 empty=true

=== Experiment 05: Set Operations ===
Union (A OR B):
  Barney Gumble Carl Carlson Frank Grimes
  Homer Simpson Lenny Leonard Moe Szyslak Otto Mann
Intersection (A AND B):
  Barney Gumble Homer Simpson Lenny Leonard
Difference (A minus B -- day-shift, non-union):
  Carl Carlson Frank Grimes

=== Experiment 06: Deduplication ===
Raw list size (with duplicates): 20
Unique departments: 7
  Engineering Executive IT Maintenance
  Research Safety Sector 7-G
Critical departments that are staffed:
  Engineering Safety Sector 7-G
Critical departments NOT staffed:
  Security
```

6. Common Mistakes

Pitfalls to avoid

- ▶ **Trying to use operator[] on a set** — sets do not support `s[i]` or `s[key]`. Use `find()` or `iterate`.
- ▶ **Expecting insert() to update an existing value** — sets store only the value, not a key-value pair. If a value already exists, `insert()` is a no-op.
- ▶ **Trying to modify an element through an iterator** — set iterators are effectively `const`. Changing an element in place would break the sort order. Erase the old value and insert the new one instead.
- ▶ **Not checking find() result before dereferencing** — if `find()` returns `end()`, dereferencing is undefined behavior.
- ▶ **Forgetting that set_union/intersection/difference require sorted input** — these algorithms require both input ranges to be sorted. Sets satisfy this automatically; vectors do not unless you sort them first.
- ▶ **Using contains() without C++20 enabled** — `contains()` requires C++20 (`-std=c++20`). Use `count()` for compatibility with older standards.

7. Reflection Questions

Answer each question in 2–3 sentences:

- ▶ 1. Why does a set not support `operator[]`? What would it mean to access the "nth" element of a set?
- ▶ 2. You need to store a list of employee IDs and quickly check whether a given ID has already been processed. Would you use a vector, a set, or a map? Explain.
- ▶ 3. What is the difference between `count()` and `find()` for membership testing? When would you prefer one over the other?
- ▶ 4. Explain why you cannot modify an element through a set iterator. How would you change an existing element?
- ▶ 5. When would you choose `unordered_set` over `set`? What property of `set` do you give up?

8. Self-Assessment: Quality Control

Before *submitting* your work review each item below. A check on every line means your lab is complete.

☐	Item
☐	Experiment 01 — duplicates are silently rejected; insert() return value checked; sorted output confirmed.
☐	Experiment 02 — three traversal styles produce correct ascending and descending output.
☐	Experiment 03 — find(), count(), and contains() all demonstrated; lower_bound and upper_bound used.
☐	Experiment 04 — erase by value, by iterator, and by condition all work correctly; clear() verified.
☐	Experiment 05 — union, intersection, and difference produce correct results; inserter used correctly.
☐	Experiment 06 — deduplication works; set_intersection identifies staffed critical departments.
☐	Code compiles with no errors and no warnings.
☐	find() return value is always checked against end() before dereferencing.
☐	No attempt to use operator[] or modify elements through iterators.
☐	Reflection questions answered in 2–3 sentences each.

Key insight

A set's two defining guarantees — **uniqueness** and **sorted order** — are enforced automatically by the container itself. You never need to write deduplication or sorting logic manually; just insert elements and the set does the rest. This makes set the right tool whenever your primary concern is "does this value exist?" rather than "what is the nth value?"